



**RAJALAKSHMI ENGINEERING COLLEGE,
CHENNAI**
Department of Artificial Intelligence & Data Science
CS23633 – Cloud Computing Laboratory

Experiment 2

PAAS in Public Cloud Using Render

AIM

To build a basic Python Flask web application and deploy it on the Render cloud platform. This experiment helps in understanding how a Platform as a Service (PaaS) allows applications to run online without manually setting up or maintaining servers.

What is PaaS?

Platform as a Service (PaaS) is a cloud service that provides everything needed to develop, deploy, and run an application. It takes care of server management, software installation, and deployment, allowing developers to focus only on writing the application.

Why Render?

Render is an easy-to-use cloud hosting platform that supports web application deployment directly from GitHub. It automatically builds and hosts the application, making deployment simple even for beginners.

PROCEDURE:

Step 1

Sign in to your GitHub account or create one if you do not already have an account.

Step 2

Create a project folder named **RenderHelloApp**.

Step 3

Inside the project folder, create the following files:

- app.py
- requirements.txt

Step 4

Write the Flask application code in the app.py file.

```
1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route('/')
6 def home():
7     return "Hello! Welcome to Modern Flask!"
8
9 @app.route('/about')
10 def about():
11     return "This application is deployed on Render!"
12
13 if __name__ == "__main__":
14     app.run(host="0.0.0.0", port=5000)
15
```

Step 5

Add the required Python packages such as **Flask** and **Gunicorn** to the requirements.txt file.

Step 6

Create a new public repository called **RenderHelloApp** on GitHub.

Step 7

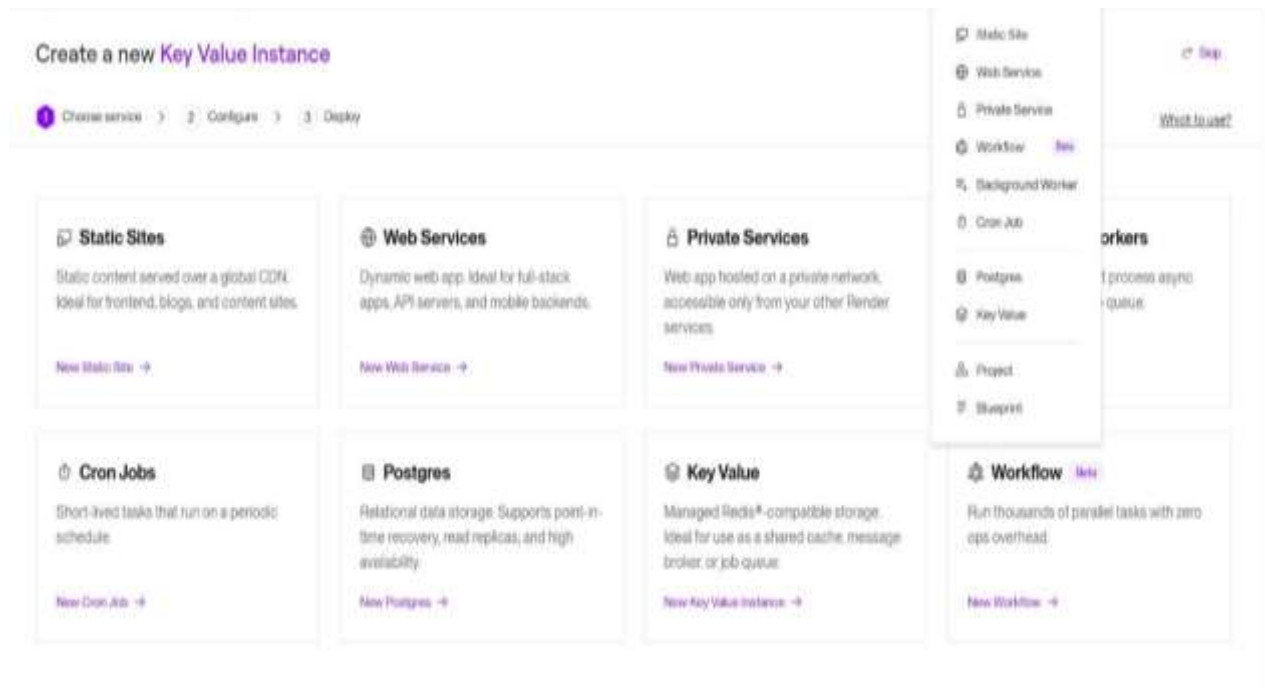
Upload both project files to the repository and commit the changes.

Step 8

Create an account on Render and connect it to your GitHub account.

Step 9

Create a new Web Service in Render.



Step 10

Select the **RenderHelloApp** repository from GitHub.

Step 11

Enter the deployment details:

- **Language:** Python
- **Branch:** main
- **Root Directory:** Leave empty
- **Build Command:** pip install -r requirements.txt
- **Start Command:** gunicorn app:app
- **Instance Type:** Free (if available)

Root Directory Optional
 If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a [monorepo](#).

Build Command
 Render runs this command to build your app before each deploy.

Start Command
 Render runs this command to start your app with each deploy.

Instance Type

For hobby projects For professional use For more power and to get the most out of Render, we recommend using one of our paid instance types. All paid instances support: • Zero Downtime	Free \$0 / month 512 MB (RAM) 0.1 CPU	Upgrade to enable more features Free instances spin down after periods of inactivity. They do not support SSH access, scaling, one-off jobs, or persistent disks. Select any paid instance type to enable these features.
	Starter \$7 / month 512 MB (RAM) 0.5 CPU	
	Pro 4 GB (RAM)	Pro Plus 8 GB (RAM)

Step 12

Click Deploy Web Service and wait for the deployment to complete.

Environment Variables
 Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more](#).

Generate

+ Add Environment Variable
+ Add from env

> Advanced

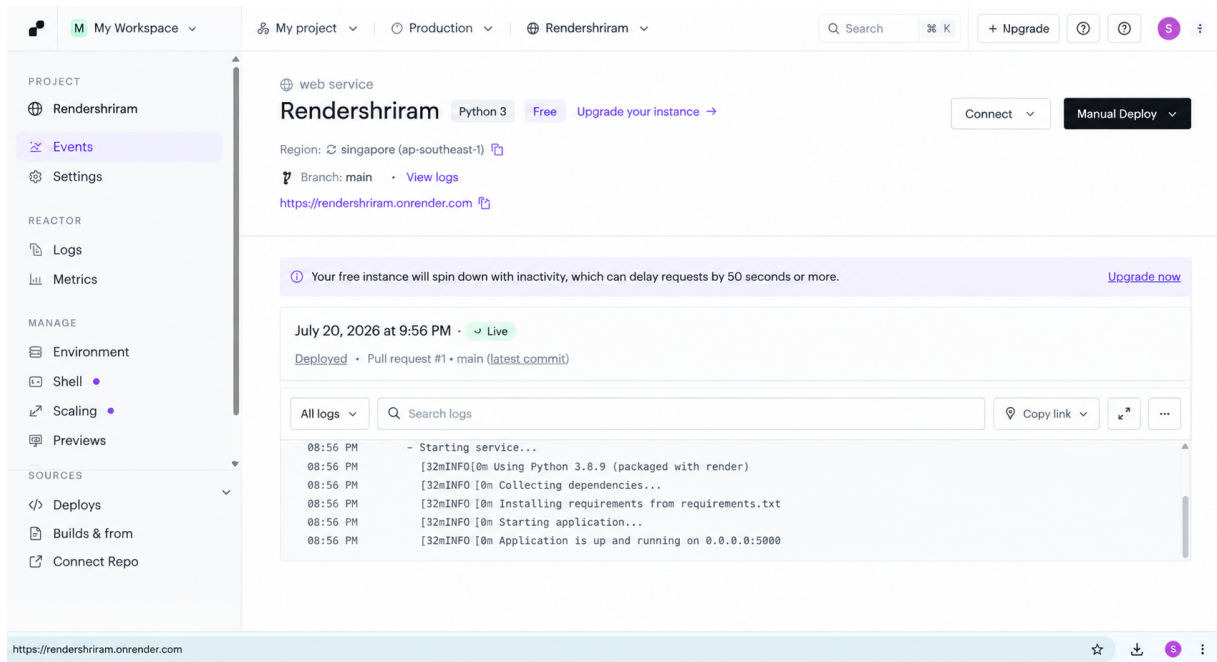
Deploy Web Service

Step 13

After deployment, open the URL generated by Render and check whether the home page is displayed correctly.

Step 14

Add **/about** at the end of the application URL to verify that the second route is also working properly.



CONCLUSION:

This experiment demonstrated how a Flask application can be deployed using Render, a Platform as a Service. I learned that PaaS simplifies application deployment by handling the server setup and hosting, allowing developers to publish applications quickly without managing infrastructure.